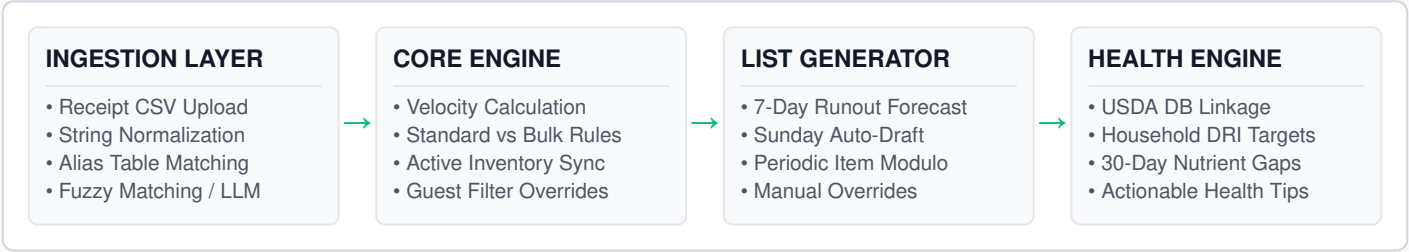


Smart Grocery Tracker & Nutrition Engine

Technical Architecture, Technology Stack & Implementation Roadmap

1. SYSTEM ARCHITECTURE

The application is structured as a decoupled, multi-tier data pipeline that processes raw bill data, tracks inventory velocity, models consumption behavior, and runs nutritional gap analysis against household demographic profiles.



2. TECHNOLOGY STACK

LAYER	TECHNOLOGIES	ROLE & JUSTIFICATION
Backend API	PythonFastAPI	Asynchronous REST API framework delivering high execution speed, automatic Pydantic validation, and OpenAPI documentation.
Database & ORM	PostgreSQLSQLAlchemyAlembic	Relational DB for line-item transactions, inventory states, and alias lookups. Alembic handles database migrations.
Data Processing	PandasNumPy	Handles sliding-window frequency calculations, quantity normalization, and 7-day inventory runout predictions.
Entity Resolution	RapidFuzzGemini API / LLM	RapidFuzz provides fast string-matching on known aliases; LLM fallback classifies unrecognized raw CSV product text.
Nutritional Data	USDA FoodData Central API	Provides exact macronutrient and micronutrient profiles per standard unit weight for canonical food items.
Frontend UI	ReactTypeScriptTailwind CSS	Type-safe user interface built with Vite, utilizing TanStack Query for caching and state management.

3. INTEGRATED IMPLEMENTATION ROADMAP

Phase 1: Database Setup, Technology Stack & Entity Normalization

Foundation

Tech Stack: Python, FastAPI, PostgreSQL, SQLAlchemy, RapidFuzz

- **PostgreSQL Schema:** Deploy tables for items, item_aliases, household, purchase_logs, inventory, and guest_events.
- **CSV Pipeline:** Build POST /bills/upload-csv with text cleanup (unit extractions like "1 Gal", "128oz").
- **Entity Resolution:** Implement RapidFuzz string matching against alias tables with LLM fallback for unrecognized entries.

Phase 2: Frequency Engine, Real-Time Inventory & Guest Handling

Core Logic

Tech Stack: Pandas, NumPy, FastAPI Services

- **Consumption Analytics:** Compute daily consumption rates with sliding windows (14–30 days standard vs. 90–180 days bulk).
- **Guest Event Allocations:** Build event endpoints to subtract guest consumption from baseline household velocity.
- **Inventory Controls:** Implement POST /inventory/adjust for spoiled food or dining-out adjustments.

Phase 3: Automated Grocery List Generation & Data Management

Automation

Tech Stack: Python Predictive Engine, REST APIs

- **Depletion Predictor:** Build GET /grocery-list/generate to run 7-day inventory forecasts for auto-adding reorders.
- **Periodic Purchase Scheduler:** Flag interval-based items (e.g., bi-weekly spinach) using date-modulo triggers.
- **Data Controls:** Support transaction line-item deletion and soft-archiving (is_active = false) for discontinued items.

Phase 4: Nutritional Intelligence & Household Health Analysis

Health Engine

Tech Stack: USDA FoodData Central API, Pandas Health Module

- **USDA DB Integration:** Map canonical items to USDA nutrition profiles for macros/micros per unit weight.
- **Household DRI Engine:** Calculate overall household Dietary Reference Intakes based on demographic metadata.
- **Dietary Gap Analysis:** Build GET /nutrition/analysis to output 30-day health feedback reports and recommendations.

Phase 5: User Interface Development & End-to-End Testing

Deployment

Tech Stack: React, TypeScript, Tailwind CSS, TanStack Query

- **Interactive UI Dashboards:** Build receipt upload preview, active inventory controls, Sunday checklist, and health panels.
- **Testing & Calibration:** Validate edge cases including long purchase gaps, duplicate receipts, and guest allocations.